Code review: Brightside Salons booking app

Sample report from an open-source demo. Brightside Salons is a fictional company; the app was written on purpose with mistakes that AI-generated code often ships with. Every finding below has a test in the repository that fails on the original code and passes on the fixed code.

Scope: the booking web app (Next.js, TypeScript, Postgres): sign-up and login, booking, cancelling, deposit payment and the payment webhook. **Not in scope:** hosting, the payment provider itself, email, load.

Summary for the owner

The app works for a customer who uses it as intended. The problems appear when someone does not: a curious customer, a script, or the payment provider resending a message. Three of them are critical and should be fixed before the app takes real bookings and payments:

Risk	What could happen to your business	Fixed
Critical	Your payment account key is visible to every visitor	Yes, and the key must be rotated
Critical	Anyone could see and cancel other customers' bookings	Yes
Critical	A fake payment message could charge cards, and a repeated one charged twice	Yes
High	Appointments were saved at the wrong time	Yes
High	Customers could set their own price	Yes
High	Unlimited password guessing	Yes
Medium	Errors were hidden instead of handled (bookings on Sundays, unpaid deposits)	Yes
Medium	One person could end up with two accounts	Yes

Three things to do now, outside the code:

1. Rotate the payment provider's secret key. The old one was shipped to browsers and must be treated as public.
2. In the payment dashboard, compare deposits with bookings and refund duplicates.
3. Correct the times of existing bookings (they were stored hours off) and merge duplicate accounts before running the new database migration.

Findings, ranked by risk

1. Your payment account key is visible to every visitor (Critical)

What could happen to your business: anyone who opens the site can copy the payment account's secret key from the page's code. With it they can act as your business at the payment provider: create charges, issue refunds, read customer payment data.

What was wrong: the "Pay deposit" button read the key from a `NEXT_PUBLIC_` variable. Next.js copies every `NEXT_PUBLIC_` value into the JavaScript that every visitor downloads. The e2e test finds the key in the production bundle of the original code.

Fix: the browser now asks our own server to start the checkout; only the server knows the key. Files that hold secrets are marked server-only, so a future mistake fails the build instead of shipping. **Test:** `tests/findings/01-secret-in-browser.test.ts`, `e2e/bundle.spec.ts`.

2. Anyone could see and cancel other customers' bookings (Critical)

What could happen to your business: a logged-in customer changes one number in a request and sees another customer's appointments and private notes, or cancels them. Booking and customer numbers are sequential, so every booking can be reached.

What was wrong: the bookings API took the customer's id from the request (`?userId=` or the body) instead of from the login session, and "cancel" did not check who owns the booking.

Fix: the customer is always the one in the signed session cookie. Asking for someone else's booking gets the same "not found" as a booking that does not exist. **Test:** `tests/findings/02-other-customers-bookings.test.ts`, and from a real browser, `e2e/other-customers-bookings.spec.ts`.

3. A fake payment message could charge cards, and a repeated one charged twice (Critical)

What could happen to your business: the address that receives payment notifications accepted messages from anyone. Payment providers also resend a notification when they are not sure it arrived, and each resend charged the customer's deposit again. Double charges mean refunds, chargeback fees and bad reviews.

What was wrong: the webhook did not verify the provider's signature, and nothing remembered which notifications had already been handled.

Fix: the signature and its timestamp are checked before anything else; each notification is recorded once; a booking is only charged while it is still waiting for its deposit; the charge request carries an idempotency key, so the provider itself refuses to charge the same deposit twice. **Test:** `tests/findings/03-payment-webhook.test.ts`.

4. Appointments were saved at the wrong time (High)

What could happen to your business: a customer books 10:00 in Chicago and the system saves 4:00 AM. Staff and customer see different times; opening hours are checked against the wrong clock.

What was wrong: the server read the picked time in its own time zone, which is UTC on most hosting platforms.

Fix: times are converted from the salon's time zone, with daylight saving handled, and shown as salon time. **Test:** `tests/findings/05-time-zone.test.ts`.

5. Customers could set their own price (High)

What could happen to your business: anyone with the browser's developer tools could book a $55 haircut for $0 or mark it as already confirmed. Bad input crashed the request instead of showing a message.

What was wrong: the booking API saved the whole request as sent.

Fix: every request is checked against a schema; unknown fields are dropped; price and status are set by the server; bad input gets a clear message naming the field. **Test:** `tests/findings/06-input-validation.test.ts`.

6. Unlimited password guessing (High)

What could happen to your business: a script could try passwords against an account as fast as the server answers. Reused or weak passwords fall quickly.

Fix: after 5 wrong passwords in 15 minutes, the account's login pauses and says when to try again. **Test:** `tests/findings/07-login-rate-limit.test.ts`.

7. Errors were hidden instead of handled (Medium)

What could happen to your business: the salon took bookings on Sundays, when it is closed. When a deposit charge failed, the app told the payment provider all was fine, so the charge was never retried and the booking stayed unpaid.

What was wrong: the build was configured to ignore code errors, and one of the ignored errors was the Sunday bug. The webhook caught every error and answered "OK".

Fix: code errors fail the build again and the Sunday rule works; the webhook answers with an error when it fails, so the provider retries (safe now that finding 3 is fixed). **Test:** `tests/findings/08-hidden-errors.test.ts`.

8. One person could end up with two accounts (Medium)

What could happen to your business: two sign-ups at the same moment, or the same email typed with a capital letter, created two accounts. Bookings split between them; password resets go to the wrong one.

Fix: the database itself now guarantees one account per email address. **Test:** `tests/findings/04-duplicate-signup.test.ts`.

Found while fixing: the key that signs login sessions had a built-in fallback value. A server deployed without its own key would accept login cookies anyone can forge. It is now required in production (`tests/session.test.ts`).

What I did not fix, and why

- **Booking numbers are still sequential.** The ownership check is the fix; random ids would only hide the numbers, and changing them changes every link already sent to customers.
- **The login pause is per account.** Someone who knows a customer's email can pause that customer's login for 15 minutes. Limiting by network address belongs in front of the app (the hosting platform or a CDN), and a CAPTCHA after a few failures is a product decision.
- **No check for bookings in the past or for two bookings at the same time.** Those are business rules (how many chairs, how far ahead) to agree with the owner, not bugs in the code as written.
- **No CSRF token.** The session cookie is `SameSite=Lax`, so browsers do not send it with cross-site POST requests, and every action is a POST. Add a token if the app ever needs to support very old browsers.
- **Password reset and email verification** do not exist in this app; they are features, not fixes.
- **A moderate advisory in a development-only tool** (`drizzle-kit` depends on an old `esbuild`). It is not part of the app that runs in production.
- **Row Level Security.** The app is the only client of its database, so access is enforced in the API. For apps where the browser talks to the database directly (Supabase), see supabase-rls-tenant-audit.

For your developer

#	Problem	Where	Commit
1	Secret in `NEXT_PUBLIC_PAYDEMO_SECRET_KEY`, used in a `"use client"` component	`src/app/bookings/pay-deposit-button.tsx`	Fix 1
2	IDOR: `userId` read from query and body; cancel without ownership check	`src/app/api/bookings/**`	Fix 2
3	Webhook: no HMAC verification, no replay window, no event dedup, no idempotency key	`src/app/api/payments/webhook/route.ts`	Fix 3
4	Check-then-insert race on sign-up; no unique index; emails not normalized	`src/app/api/auth/signup/route.ts`	Fix 4
5	Date and time parsed with `new Date()` in the server's zone; `getHours()` for opening hours	`src/lib/booking-time.ts`	Fix 5
6	`...body` spread into the insert (mass assignment); no schema validation	`src/app/api/bookings/route.ts`	Fix 6
7	No rate limit on credentials login	`src/app/api/auth/login/route.ts`	Fix 7
8	`ignoreBuildErrors: true` hiding `closedDays.includes(day.toString())`; `catch {}` returning 200	`next.config.ts`, webhook, cancel	Fix 8

How to check the work:

```
npm ci
npm test                 # the fixed code: all tests pass
npm run test:before      # the same tests against tag vibe-coded: the finding tests fail
npm run test:e2e         # production build in a real browser
git log --oneline vibe-coded..main   # one commit per fix, each naming its test
```

Each fix commit changes the code for one finding only and names the test that proves it. The tests call the real route handlers against a real Postgres engine (PGlite, in memory), not mocks of the database.